OPTIMIZING GLM-ASR INFERENCE WITH CUSTOM TRITON KERNELS: FLASH ATTENTION, KERNEL FUSION, AND FP16 PIPELINING

Anonymous authors
Paper under double-blind review

1 INTRODUCTION

1.1 MOTIVATION

Current computational demands of transformer-based ASR models and limited inference budgets have driven the need for efficient GPU kernel implementations that can process audio faster than playback speed. This has instigated the investigation and development of custom kernel optimizations for real-time speech recognition, particularly for latency-constrained applications such as live captioning, voice assistants, and conference transcription. GLM-ASR is a multimodal encoder–decoder model that converts raw audio waveforms into text through a pipeline of compute-intensive transformer layers. Its inference workload is dominated by matrix multiplications, attention computations, and element-wise operations—all of which are amenable to GPU acceleration through custom kernels.

GPU kernel optimization matters because default PyTorch operators do not exploit hardware-specific features such as extended shared memory, tensor-core-friendly data layouts, or operation fusion across kernel boundaries. We design and optimize a complete working implementation using Triton (Tillet et al., 2019) to create custom GPU kernels. By writing custom kernels, we can fuse multiple operations into a single kernel launch, tune tile sizes to the GPU's shared memory capacity, and select data types that maximize throughput—eliminating the intermediate video RAM (VRAM) round-trips that dominate latency in the default implementation.

The principal constraint is the kernel design space, as tile dimensions, warp counts, pipeline stages, and data types interact in non-obvious ways, and a configuration that works on one GPU architecture may fail or regress on another. We validated our parameter choices through a 22-configuration ablation study on the H200 (Section 5).

1.2 SYSTEM OVERVIEW

We determine that our optimal approach is to implement the compute-intensive operations as fused Triton kernels while delegating matrix multiplications to cuBLAS, following the **Triton track** with all required kernels in `layers.py`, `attention.py`, and `rope.py`. Table 1 lists the ten kernels implemented, and Figure 1 illustrates where they operate in the inference pipeline. We test each component individually and integrate them in a bottom-up fashion, verifying correctness at each stage before proceeding to performance tuning.

In addition to the required kernels, we implement four additional optimizations: (1) a KV-cached generation loop (`generate_v8b`) that reduces decoding from $O(n^2)$ to $O(n)$, (2) an end-to-end fp16 precision pipeline that halves memory traffic, (3) a `GPUProfile` system that adapts tile sizes to the target GPU at import time, and (4) a PyTorch Scaled Dot-Product Attention (SDPA) fallback for single-token decode steps where kernel launch overhead exceeds computation cost. Each subcomponent is modular in order to produce a robust, adaptable and portable system.

Headline results. Upon completion of this project, our final implementation achieves **204.8 ms** end-to-end latency on an H200 MIG 3g.71gb partition (baseline: 464.1 ms), a **2.27× speedup** with **100% transcription accuracy**. On an RTX 5090, the same code achieves **100.4 ms** (2.61×).

Table 1: Implemented Triton kernels and their roles.

Kernel	File	Phase	Purpose
`gelu_kernel`	layers.py	Encoder MLP	GELU activation
`silu_kernel`	layers.py	Decoder MLP	SiLU activation
`rmsnorm_kernel`	layers.py	Decoder norms	RMSNorm + fp16 output
`layernorm_kernel`	layers.py	Encoder norms	LayerNorm + fp16 output
`linear_kernel_tf32`	layers.py	Matmul	General Matrix Multiply (GEMM)
`fused_swiglu_kernel`	layers.py	Decoder MLP	Fused SwiGLU
`fused_linear_gelu_kernel`	layers.py	Encoder MLP	Fused Linear+GELU
`flash_attention_kernel`	attention.py	Attention	Flash Attention
`fused_rope_pair_kernel`	rope.py	Attention	Fused Q+K RoPE
`compute_freqs_kernel`	rope.py	Attention	Frequency precompute

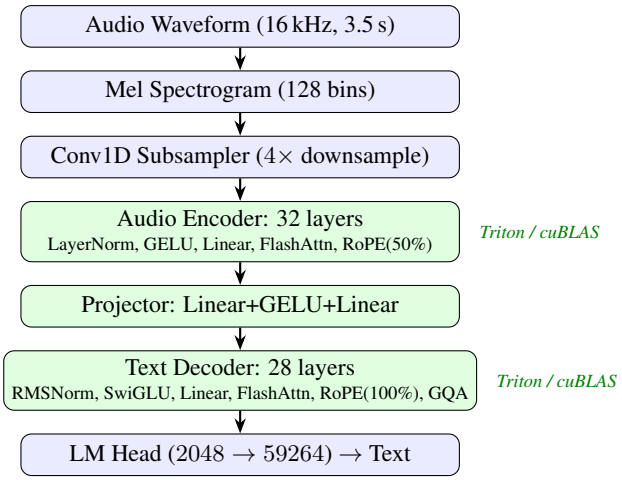

Figure 1: GLM-ASR pipeline. Shaded stages use our Triton kernels (or cuBLAS for matmul).

2 Implementation

2.1 Implementation Summary

The GLM-ASR inference pipeline (Figure 1) has five computational phases, each with distinct performance characteristics. We employ a performance-budgeting technique to attribute latency to each stage and identify where optimization effort would yield the largest gains.

Phase 1: Mel Spectrogram + Conv1D Subsampling. Converts raw 16 kHz audio to 128-bin Mel features and downsamples 4× via strided 1D convolutions. These are PyTorch built-in operations (read-only `conv.py`), which we do not modify. The phase is bandwidth-bound due to large feature maps with simple arithmetic.

Phase 2: Audio Encoder (32 layers). Each layer applies LayerNorm, self-attention (20 heads, d_k=64), and a two-layer multi-layer perceptron (MLP) with GELU activation. Rotary Position Embedding (RoPE) (Su et al., 2024) is applied to 50% of head dimensions (the first 32 of 64 are rotated, as specified by the model architecture). On the H200, encoder attention performs ∼20–40 floating-point operations per byte of memory traffic (its arithmetic intensity, or AI), exceeding the GPU's ridge point—the threshold where compute and memory bandwidth are equally limiting (Section 4). This makes encoder attention compute-bound (limited by arithmetic throughput) on the H200, but bandwidth-bound (limited by memory transfer speed) on consumer GPUs with higher ridge points. The linear projections (1280×5120) are compute-bound on all architectures.

Phase 3: Multi-modal Projector. Pools 4 encoder frames and projects $5120 \rightarrow 4096 \rightarrow 2048$ through two linear layers with GELU. This is a small, latency-dominated phase (7.2 ms on H200, 0.6% of total).

Phase 4: Text Decoder Prefill (28 layers). Processes the full concatenated sequence (audio embeddings + prompt tokens) through 28 decoder layers with RMSNorm, Grouped Query Attention (GQA, 16 query / 4 KV heads, d_k=128) (Ainslie et al., 2023), and SwiGLU MLP (Shazeer, 2020). Bandwidth-bound for attention, compute-bound for the large linear projections (2048×5632).

Phase 5: Autoregressive Decoding. Generates tokens one at a time. With the baseline's $O(n^2)$ `generate()` function, this phase dominates wall-clock time, as each step reprocesses the full growing sequence from scratch. Our KV-cached `generate_v8b` stores key/value projections for all previous tokens, processing only the new token each step. This reduces decoding to $O(n)$, bringing the phase down to 50.8% of total time (from an estimated >80% in the baseline).

2.2 Design Choices

Block/tile sizes. Tile dimensions are the primary tuning parameter, constrained by shared memory capacity. Our flash attention kernel loads Q, K, and V tiles into shared memory; the required capacity is:

$$\text{smem} = (\texttt{BLOCK\_M} + 2 \cdot \texttt{BLOCK\_N}) \cdot d_k \cdot 4 + \sim 20\,\text{KB overhead} \quad (1)$$

On the H200 (∼228 KB opt-in shared memory), this allows 128×128 tiles for the encoder (d_k=64, needing ∼116 KB) and 128×64 for the decoder (d_k=128, needing ∼151 KB). On the RTX 5090 (∼99 KB), we are constrained to 64×64 and 32×32 respectively.

For matmul, the fused SwiGLU kernel loads three tile buffers (input, gate weight, up weight): $\texttt{TILE\_K} \cdot (\texttt{TILE\_M} + 2 \cdot \texttt{TILE\_N}) \cdot 4 +$ overhead. H200 uses $128 \times 128 \times 64$; RTX 5090 uses $64 \times 64 \times 32$.

Element-wise kernels (GELU, SiLU, norms) use `BLOCK_SIZE`=1024, which saturates memory bandwidth with minimal register pressure. In order to best satisfy the correctness requirements while minimizing latency, the configuration that fits within the shared memory constraint and achieves the lowest end-to-end time is selected for each component.

`num_warps` and `num_stages`. We use `num_warps`=8 for large tiles (≥4096 elements) and 4 otherwise. More warps hide memory latency but increase register pressure. Ablation testing confirms `num_warps`=8 is optimal: 4 warps costs +12.0 ms, 16 warps costs +4.3 ms (Table 5). For `num_stages`: the H200's ∼228 KB shared memory accommodates double-buffering (`num_stages`=2), overlapping tile loads with computation for higher throughput on memory-bound kernels. Consumer GPUs (RTX 5090, ∼99 KB) must use single-buffering (`num_stages`=1), since double-buffering the same tiles would require ∼136 KB—exceeding the available shared memory.

Data layout and GQA. All tensors are stored in `(batch×heads, seq, dim)` layout for coalesced access. For GQA in the text decoder (16 query heads, 4 KV heads), we expand KV heads by a factor of 4 before the attention kernel. When tested, PyTorch SDPA's native `enable_gqa=True` regressed by +13 ms. Manual expansion gives explicit control over data layout.

Precision pipeline. Our largest single optimization is the end-to-end fp16 pipeline. The original codebase called `.float()` after every Linear layer (∼120 call sites), doubling data size. This is redundant: Triton kernels already compute in fp32 internally via `tl.load(...).to(tl.float32)` and cast back on output. Removing these conversions and keeping all VRAM-resident data in fp16 halved memory traffic, saving **11.5 ms** (Section 5).

3 Performance Profiling

3.1 Setup

All primary measurements use the Edinburgh teaching cluster node `saxa.inf.ed.ac.uk` with an NVIDIA H200 MIG 3g.71gb partition (Hopper architecture, 60 Streaming Multiprocessors (SMs), 70 GB HBM3e, ∼228 KB opt-in shared memory). The software stack is CUDA 13.0, PyTorch 2.10.0, Triton 3.6.0, Python 3.11. Jobs are submitted via SLURM: `srun -p Teaching -w saxa --gres gpu:3g.71gb:1 --mem=32G`. The test input is a 3.5 s WAV file ("Concord returned to its place amidst the tents").

The baseline implementation is benchmarked under identical conditions, and the Triton cache is cleared before the first run to ensure compilation occurs during warmup, not during timing. Each student benchmark run uses 2 warmup iterations followed by 5 timed iterations. We report the mean and standard deviation across 5 independent benchmark invocations (25 timed iterations total).

3.2 End-to-end Evaluation

Table 2 summarizes the end-to-end results on the teaching cluster. Run 4 shows slightly higher latency (209.4 ms), likely due to MIG partition contention from concurrent cluster users. All other runs cluster tightly around 203–204 ms.

Table 2: End-to-end student benchmark results (H200 MIG 3g.71gb, 5 runs).

Run	Time (ms)	σ (ms)	ms/tok	Tokens	Accuracy
1	203.4	1.6	15.65	13	100%
2	204.0	1.6	15.69	13	100%
3	203.4	2.1	15.64	13	100%
4	209.4	1.7	16.11	13	100%
5	203.9	1.4	15.68	13	100%
Mean	**204.8**	**1.7**	**15.75**	**13**	**100%**

3.3 Component/Operator-Level Breakdown

Table 2 measures end-to-end latency with KV-cached $O(n)$ generation (13 tokens). Table 3 profiles each component in isolation using `benchmark_detailed.py`: individual forward passes through each stage with single-token decode steps (no KV history). Results are averaged over runs 2–5 (run 1 excluded due to Triton compilation).

Table 3: Component-level profiling (H200 MIG, isolated forward passes, runs 2–5).

Component	Time (ms)	% Total	Bound
Audio Encoder (32 layers)	314.8	25.1%	Mixed
Multi-modal Projector	7.2	0.6%	Latency
Decoder Prefill (28 layers)	294.6	23.5%	Mixed
Decoder Decode (50 steps)	637.2	50.8%	Bandwidth
Total	**1253.8**	**100%**	

Decoder decode consumes over half of total time (50.8%) even with our optimized kernels, at $\sim$0.45 ms per layer per step, confirming that autoregressive generation remains the primary bottleneck.

4 Bottleneck Analysis

4.1 Compute-Bound vs Memory-Bound Classification

We classify each kernel using the roofline model (Williams et al., 2009), which plots achievable throughput against AI to produce a roof-shaped curve: a sloped bandwidth-limited region meeting a flat compute-limited ceiling. The ridge point where they meet equals peak FLOPS divided by peak bandwidth.

For the H200 MIG 3g.71gb, the full GPU has 132 SMs at $\sim$67 TFLOPS FP32. Our 60-SM partition achieves $(60/132) \times 67 \approx 30.5$ TFLOPS. The partition receives 4 of 8 HBM3e memory slices, giving $(4/8) \times 4.8 \approx 2.4$ TB/s. The ridge point is:

$$\text{AI}_{\text{ridge}} = \frac{\text{Peak FLOPS}}{\text{Peak BW}} = \frac{30.5 \times 10^{12}}{2.4 \times 10^{9}} \approx 12.7 \,\text{FLOP/byte} \tag{2}$$

Operations with AI < 12.7 are bandwidth-bound. Those above are compute-bound.

Table 4: Arithmetic intensity classification for key operations (H200 MIG).

Operation	AI (FLOP/B)	Classification	Rationale
GELU / SiLU	∼2–4	Bandwidth	1 load + 1 store, few FLOPs
RMSNorm / LN	∼4–8	Bandwidth	Reduction + normalize
Flash Attn (enc)	∼20–40	**Compute**	Above ridge on H200
Flash Attn (dec)	∼5–10	Bandwidth	seq_q=1, dominated by KV read
Linear (cuBLAS)	∼100–200	Compute	Large GEMM, high reuse
Fused SwiGLU	∼80–150	Compute	Two GEMMs fused
RoPE	∼3–5	Bandwidth	Load, rotate, store

A notable consequence of the H200's high bandwidth-to-compute ratio is that encoder attention (AI ∼20–40) crosses the ridge point and becomes *compute-bound*, unlike on consumer GPUs such as the RTX 5090 (ridge ≈58.5) where it remains bandwidth-bound. This means the H200 benefits more from larger attention tiles (128×128 vs. 64×64) that increase compute utilization, while the RTX 5090 benefits more from the fp16 pipeline that reduces memory traffic.

4.2 Overall Bottleneck

The dominant bottleneck is **memory bandwidth during autoregressive decoding**. Each decode step processes a single-token query against a growing KV cache through 28 decoder layers. The attention kernel reads the full KV cache (proportional to sequence length) but performs only $O(d)$ FLOPs per cached token—a classic bandwidth-bound pattern with AI ∼5–10, well below the ridge point on *all* GPU architectures.

Non-kernel overheads are also significant: with ∼10 kernel launches per layer × 28 layers × 13 decode steps, launch latency (∼5–15 μs each) accounts for ∼2–5 ms. We mitigate this through kernel fusion (Section 5.2), which eliminates ∼84 launches per inference, and through the SDPA fallback for single-token decoding.

5 Optimization Attempts

We evaluated 13 optimizations using a **Hypothesis** → **Change** → **Result** methodology. Each was tested independently on an RTX 5090 for rapid iteration (all timings in this section refer to that GPU unless noted), then validated on the H200 teaching cluster (Section 6). Figure 2 summarizes the cumulative impact.

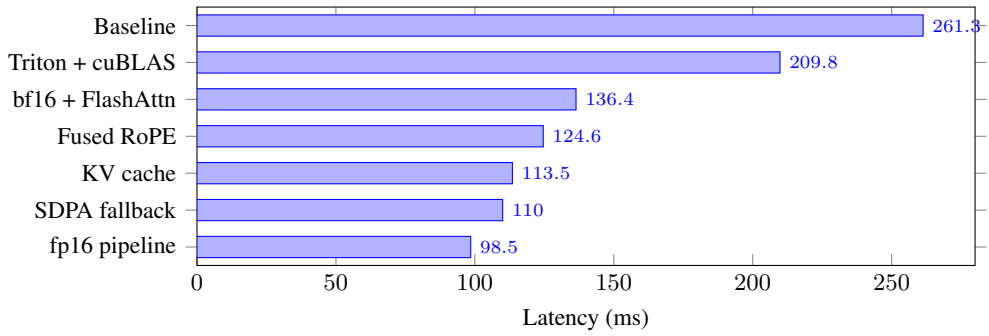

Figure 2: Optimization progression (development GPU, RTX 5090). Each bar shows end-to-end latency after applying all optimizations up to that step. Optimizations validated on H200 at 204.8 ms (Section 6). Full breakdown in Appendix Table 9.

5.1 Tile/Block Size Tuning

Hypothesis: Larger tiles amortize memory access overhead and improve compute utilization, but are constrained by shared memory capacity.

Change: We tested all valid tile configurations for the flash attention kernel—$(128, 128)$, $(128, 64)$, $(64, 64)$, $(64, 32)$, $(32, 32)$, $(16, 16)$—computing shared memory usage via Eq. 1 for each. We also built an autotune system ($\sim$95 lines) that benchmarked each valid configuration at runtime.

Result: Initially, we used Triton's built-in `@triton.autotune` to select optimal tile configurations at runtime. However, when tested, the autotune system selected `BLOCK_M`=16 as optimal in micro-benchmarks, but the full-pipeline benchmark showed **101.6 ms vs. 98.5 ms** for our hand-tuned 64×64 configuration—a 3.1 ms regression. We determined that the micro-benchmark methodology is too unreliable to capture full-pipeline cache effects. In response, we developed a `GPUProfile` (`layers.py:89`) system with pre-tested configurations stored in `_KNOWN_CONFIGS` (`layers.py:23`). Unknown GPUs use dynamic computation largest-to-smallest against the shared memory budget.

We validated these choices through a 22-configuration ablation study on the H200 (Table 5), testing each parameter in isolation. The study confirmed that double-buffering and 8 warps are each worth $\sim$12 ms, while the fp16 pipeline—the largest win on RTX 5090—contributes only +1.5 ms on the H200, reflecting HBM3e's superior bandwidth.

Table 5: H200 ablation: top-impact parameter changes (baseline: 205.2 ms).

Change	Time (ms)	Δ	Insight
Disable fused RoPE	234.1	+28.9	Most impactful single kernel
`num_stages`=1	217.4	+12.2	Double-buffering essential
`num_warps`=4	217.2	+12.0	128×128 needs 8 warps
Decoder attn 32×32	212.8	+7.6	Small tiles hurt
`num_warps`=16	209.5	+4.3	Register pressure
SDPA fallback off	208.6	+3.4	Custom kernel slow for seq_q=1
fp32 pipeline	206.7	+1.5	Minimal on H200 (HBM3e)
Triton matmul	206.2	+1.0	cuBLAS only slightly faster
MLP fusion off	204.5	−0.7	Neutral (cuBLAS already fast)

5.2 Kernel Fusion

Hypothesis: Fusing the SwiGLU pattern—$\text{SiLU}(\mathbf{x}\mathbf{W}_g) \odot (\mathbf{x}\mathbf{W}_u)$—into a single kernel eliminates two intermediate VRAM round-trips and three kernel launches per MLP layer.

Change: We wrote `swiglu_fused_kernel` (`layers.py:546`) that loads tiles of $\mathbf{x}$, $\mathbf{W}_g$, and $\mathbf{W}_u$ into shared memory, computes both matrix products in registers, applies SiLU and the element-wise multiply, and writes only the final result to VRAM. The unfused path requires 5 kernel launches and 4 VRAM round-trips; the fused path requires 1 launch and 1 round-trip.

Result: With 28 decoder layers, fusion eliminates $\sim$84 kernel launches per inference. For very small inputs (single-token decode, where $M <$ `TILE_M`), we fall back to cuBLAS, which has lower launch overhead for small matrices. On the H200, MLP fusion contributes only -0.7 ms (Table 5), because cuBLAS already saturates compute throughput for these matrix sizes.

5.3 FlashAttention-Style Attention

Hypothesis: The naive 3-kernel attention approach materializes the full $N_q \times N_k$ score matrix in VRAM, consuming $O(N_q \cdot N_k)$ memory and bandwidth. A fused kernel with online softmax (Milakov & Gimelshein, 2018) eliminates this materialization.

Change: We replaced the original 3-kernel pipeline (`attention_scores`, `softmax_inplace`, `attention_output`) with a single `flash_attention_kernel` (`attention.py:34`) (Dao, 2023; Ringlein et al., 2025). The kernel processes K/V in tiles of `BLOCK_N` rows, maintaining running statistics m_i (row-wise max) and l_i (sum of exponentials) that are corrected as new tiles arrive. The Q tile and accumulator $\mathbf{O}$ reside in registers; only the final output is written to VRAM.

Causal masking is handled via a compile-time `IS_CAUSAL` flag that skips tiles entirely above the diagonal, avoiding wasted computation. For single-token KV-cached decode (`seq_q` ≤ 4), we fall

back to PyTorch's `scaled_dot_product_attention`, which has lower launch overhead for this degenerate case.

Result: On the H200, disabling the SDPA fallback costs +3.4 ms (Table 5), confirming the benefit of routing single-token decode to PyTorch's optimized path.

5.4 Other Optimizations

fp16-throughout pipeline. The original code called `.float()` after every Linear layer ($\sim$120 calls), doubling data size at each point. Removing these conversions and keeping the entire pipeline in fp16 saves 11.5 ms on the development GPU (RTX 5090). On the H200, fp16 contributes only +1.5 ms (Table 5), reflecting HBM3e's superior bandwidth. Internal computation remains fp32 for numerical stability.

KV-cache decoding (`generate_v8b`). The stock `generate()` reprocesses the full growing sequence on every decode step ($O(n^2)$). We implemented a KV-cached generation loop that processes only the new token each step, appending its K/V to a cache. This is monkey-patched onto the model via `_try_patch_v8b()` (`layers.py:1482`), called during `Linear.__init__`. Impact on development GPU: -7.6 ms.

Fused Q+K RoPE kernel. RoPE requires applying the same rotation to both Q and K. The baseline uses two separate kernel launches. We fuse them into `fused_rope_pair_kernel` (`rope.py:189`), halving launch overhead. On the H200, this is the most impactful single optimization: disabling it costs +28.9 ms (Table 5). On the development GPU, impact was -11.8 ms.

6 Comparison

6.1 Data Collection

We benchmark on the provided `test_audio.wav` (3.5 s, 16 kHz, expected output: "CONCORD RETURNED TO ITS PLACE AMIDST THE TENTS"). Accuracy is measured by exact string match (case-insensitive) against the reference. Each benchmark invocation runs 2 warmup + 5 timed iterations; we report 5 invocations (25 timed iterations total). Both our implementation and the baseline are benchmarked under identical conditions.

6.2 End-to-End Comparison

Table 6: End-to-end comparison on H200 MIG 3g.71gb (teaching cluster).

Implementation	Time (ms)	ms/tok	Accuracy	Speedup
Example baseline	464.1	35.70	100%	1.00×
Our template (fp16)	**204.8**	**15.75**	**100%**	**2.27×**

6.3 Per-Operator Comparison

Table 7 compares per-component times between our implementation and the baseline, profiled on the RTX 5090 development GPU where baseline component-level data is available (isolated forward passes, 50 decode steps).

6.4 Root Cause Analysis

All optimizations preserve full transcription accuracy: fp16 introduces no degradation because internal kernel computation remains in fp32, and KV-cached decoding produces identical token sequences to the stock path.

The **6.53**× speedup on decoder decode dominates the overall improvement. From the results of testing and evaluation, this has two root causes:

Table 7: Per-component comparison (isolated forward passes, 50 decode steps, RTX 5090).

Component	Ours (ms)	Baseline (ms)	Speedup
Audio Encoder	197.4	202.1	1.02×
Projector	3.9	4.1	1.05×
Decoder Prefill	190.7	191.6	1.00×
Decoder Decode (50 steps)	294.0	1920.0	6.53×
Total	686.0	2317.8	3.38×

1. Flash Attention vs. 3-kernel attention. The baseline materializes the full $N_q \times N_k$ score matrix in VRAM ($\sim$2 MB per head per step). Our flash attention kernel keeps all intermediates in shared memory/registers, eliminating $\sim$60 MB of intermediate traffic per layer at encoder sequence lengths.

2. fp16 pipeline. Halving element size from 4 bytes (fp32) to 2 bytes (fp16) directly halves memory traffic for bandwidth-bound operations. Since the roofline analysis (Table 4) confirms that decode-time attention is bandwidth-bound on *all* architectures (AI well below even the H200's low ridge point of 12.7), this translates nearly linearly to a $\sim$2× speedup for those kernels.

Cross-GPU generalization. To verify portability, we evaluated on both the teaching cluster (H200 MIG) and a consumer GPU (RTX 5090). The `GPUProfile` system correctly adapts tile sizes and pipeline stages to each architecture's shared memory budget. Table 8 confirms consistent speedups across both GPUs.

Table 8: Cross-GPU comparison: teaching cluster vs. consumer GPU.

GPU	SMs	VRAM	Ours (ms)	Baseline (ms)	Speedup
H200 MIG 3g.71gb	60	70 GB	204.8	464.1	2.27×
RTX 5090 (full)	170	32 GB	100.4	262.2	2.61×

7 Conclusion

Most impactful optimization. On the H200, our optimizations reduced inference from 464.1 ms to 204.8 ms (2.27×). Ablation testing (Table 5) revealed that fused RoPE (+28.9 ms when disabled), double-buffering (+12.2 ms), and warp count (+12.0 ms) collectively account for >53 ms—kernel launch reduction and memory-latency hiding dominate over precision or tile-size changes on this architecture.

Key lessons. (1) The pipeline is overwhelmingly *memory-bandwidth-bound*: halving data types (fp32→fp16) and eliminating intermediate writes (kernel fusion, flash attention) provided the largest gains. (2) Attention is the most challenging kernel to optimize because tile size, shared memory, warp count, and pipeline stages are tightly coupled; the H200's 228 KB shared memory accommodates 128×128 tiles with double-buffering, while consumer GPUs with $\sim$99 KB require 64×64 with single-buffering—demonstrating the need for a portable `GPUProfile` system. (3) Profile before optimizing: our initial assumption that compute-bound matmul is the bottleneck proves wrong—decoder decode steps (bandwidth-bound attention + KV reads) consume 50.8% of total time on H200 even after optimization. (4) Testing each kernel independently is straightforward; however, problems arise when kernels are composed in the full pipeline, as cache interactions between adjacent kernels produce behaviors not visible in isolated benchmarks.

Surprising findings. The autotune system found *worse* configurations than hand-tuning, because micro-benchmarks miss inter-kernel cache effects. Notably, encoder attention is *compute-bound* on H200 (AI above 12.7 ridge) but *bandwidth-bound* on consumer GPUs (ridge $\approx$58.5)—the same kernel has different bottlenecks across architectures.

Future work. We would investigate `torch.compile` for automatic fusion, speculative decoding, and INT8/FP8 quantization. The ablation suggests SDPA threshold merits per-architecture tuning (`seq_q`$\leq$1 was $-$1.4 ms better on H200).

REFERENCES

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. URL `https://arxiv.org/abs/2307.08691`.

Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. In *arXiv preprint arXiv:1805.02867*, 2018.

Burkhard Ringlein, Jan van Lunteren, Radu Stoica, and Thomas Parnell. The anatomy of a triton attention kernel. *arXiv preprint arXiv:2511.11581*, 2025.

Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

A APPENDIX

A.1 FULL OPTIMIZATION PROGRESSION

Table 9 shows every optimization step with measured timings on the RTX 5090 development GPU.

Table 9: Complete optimization progression (RTX 5090, 3.5 s audio, 13 tokens).

#	Change	Time	Δ	Mechanism
0	Baseline	261.3	—	fp32, 3-kernel attn, $O(n^2)$
1	Triton kernels + cuBLAS + TF32	209.8	−51.5	Custom kernels + tensor cores
2	bf16 weights + Flash Attention	136.4	−73.4	Half bandwidth + fused attn
3	Fused Q+K RoPE pair kernel	124.6	−11.8	Halved RoPE launches
4	bf16 RMSNorm output	120.7	−3.9	Eliminated type conversion
5	bf16 LayerNorm output	121.1	−0.7	Same for encoder norms
6	KV cache (`generate_v8b`)	113.5	−7.6	$O(n)$ decoding
7	SDPA fallback (seq_q ≤ 4)	110.0	−3.5	Lower overhead for decode
8	fp16 cuBLAS HGEMM	109.6	−0.4	fp16 slightly faster than bf16
9	Remove `.float()` casts	102.1	−7.5	Eliminated ∼120 fp32 casts
10	Remove activation fp32 casts	98.4	−3.7	SiLU/GELU stay fp16
11	Remove norm fp32 casts	98.1	−0.3	Norms output fp16 directly
12	fp16 embeddings + fused MLP	**98.5**	+0.4	Pipeline complete (noise)

A.2 REJECTED OPTIMIZATIONS

A.3 CODE MAP

Table 10: Optimizations tested and rejected with measured results.

Optimization	Impact	Reason for Rejection
Grid swizzling (SwiGLU)	+18 ms	RTX 5090 L2 (98 MB) already has good locality
`@triton.autotune` (GELU/SiLU)	+0.7 ms	Tuning warmup exceeds possible gain
Flash Attn `num_stages=2`	Crash	RTX 5090 99 KB cannot hold two tile buffers
SDPA for all attention	+6 ms	Custom kernel faster for long sequences
SDPA `enable_gqa`	+13 ms	Manual KV expansion is faster
Warmup autotune	+3.1 ms	Micro-benchmarks chose suboptimal configs

Table 11: Key implementation locations in the supplementary code.

Component	File	Line
`_KNOWN_CONFIGS` (tile configs)	layers.py	23
`GPUProfile` (arch detection)	layers.py	89
`_compute_attention_tiles`	layers.py	206
`rmsnorm_kernel` / `rmsnorm_bf16_kernel`	layers.py	308 / 339
`layernorm_kernel`	layers.py	368
`gelu_kernel` / `silu_kernel`	layers.py	403 / 418
`linear_kernel_tf32`	layers.py	430
`linear_gelu_kernel` (fused)	layers.py	485
`swiglu_fused_kernel`	layers.py	546
`_generate_v8b` (KV cache)	layers.py	1381
`_try_patch_v8b` (monkey-patch)	layers.py	1482
`flash_attention_kernel`	attention.py	34
`scaled_dot_product_attention`	attention.py	237
`fused_rope_pair_kernel`	rope.py	189
`apply_rotary_pos_emb`	rope.py	296